

Efficient VLA-JEPA for Robotic Manipulation

CSI 5341: efficiency evaluation

Noah Sprenger Fernando Nogueira

2026-09-25

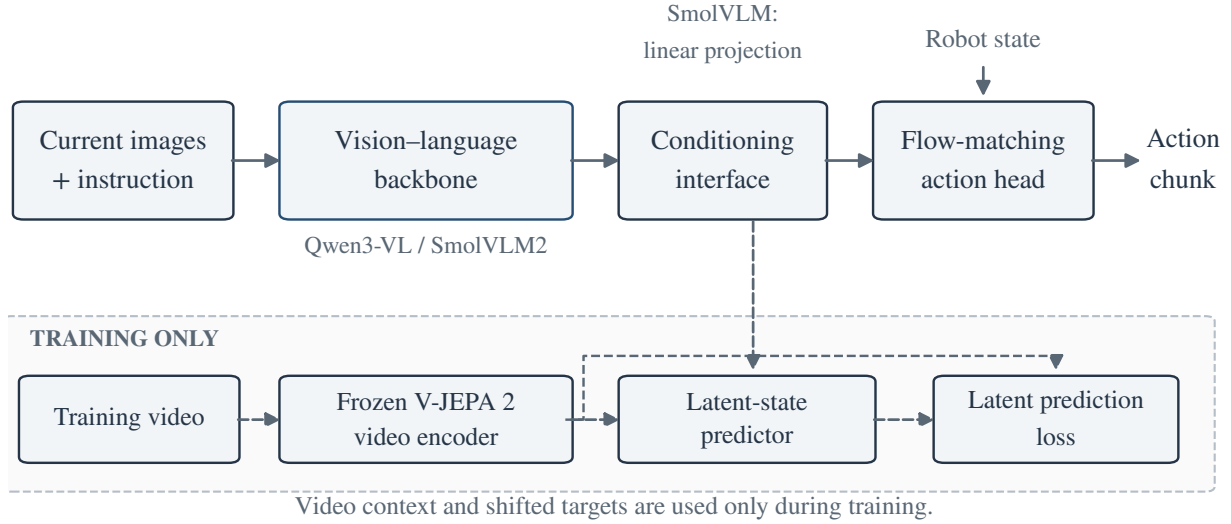
Abstract

We study whether reduced-precision inference and a smaller vision-language backbone lower VLA-JEPA’s deployment cost while retaining manipulation success. We compare a pretrained BF16 policy, two decoder-quantized variants, and a SmolVLM2-500M variant requiring reduced fine-tuning. The protocol combines matched LIBERO-Spatial trials, inference measurements, and a training-budget study. The completed protocol evaluates 500 reserved trials per configuration. Success rates are B16 95.0%, Q8 94.8%, Q4 95.8%, S500 2.2%. Efficiency and paired uncertainty estimates characterize the resulting deployment trade-offs.

1 Study design

The baseline is `1erobot/VLA-JEPA-LIBERO` [1]. Q8 and Q4 quantize its trained Qwen language-decoder linear layers using LLM.int8 [5] (outlier threshold 6) and NF4 with double quantization [6], respectively; the vision tower, embeddings, action head, and world model remain BF16. Quantization happens *after* policy checkpoint restoration. S500 replaces the backbone with SmolVLM2-500M-Video-Instruct [3] and projects its decoder representation into the existing conditioning interface. The adaptation recipe freezes SmolVLM and the V-JEPA 2 encoder [4], training the adapter and compatible action/world predictors. Qwen is evaluation-only. Dream-RSI is deferred.

A Policy architecture and supervision



B Matched deployment comparison

● Qwen · BF16 ● Qwen · 8-bit ● Qwen · 4-bit ● SmolVLM · BF16

Figure 1: Architecture and comparison arms. Solid arrows denote the inference path; dashed connections denote training supervision. The action head also receives robot state. The schematic omits the action-training loss for clarity. All configurations use identical observation/action settings and world-model loading policy.

2 Evaluation methodology

E1: manipulation. We use all ten LIBERO-Spatial tasks [2], batch size one, matched initial states and per-episode random seeds. Development uses ten original initial-state vectors per task, excluded by exact hash from the official pruned states. Final evaluation uses all fifty official pruned states per task: 500 trials per configuration. The committed manifest records state-file and state-vector hashes. Development scores are diagnostic and are not pooled with final benchmark scores.

We estimate task-macro-average success and task-level rates with 95% intervals and task-stratified paired bootstrap differences from B16. Intervals describe uncertainty on this fixed task suite, not generalization to unseen tasks. System failures remain separate from valid unsuccessful robot trials. Final trials do not enter checkpoint selection.

E2: efficiency. On one GPU (NVIDIA GeForce RTX 3090), we time a fixed observation/instruction bank with twenty warm-up predictions and 500 measured action-chunk predictions per repetition (three repetitions). CUDA synchronization brackets each measurement. Outcomes are median/p95 neural-only and observation-to-action latency, peak allocated/reserved GPU memory, parameter count, and serialized weight size. Timings exclude simulator execution and buffered-action retrieval; neural-only timing additionally excludes tokenization and image preprocessing. Memory reduction is not assumed to imply a speedup.

E3: adaptation. We split demonstrations by whole trajectories, reserving 10% for validation. The completed 500-step pilot established execution feasibility and resource cost. The frozen budget is 10,000 optimizer steps (seed 42), with selection checkpoints at 1,500, 5,000, and 10,000 steps from one continuing run. Selection minimizes sample-weighted held-out total loss; ties favor fewer steps. The pilot projects approximately ten GPU-hours including validation and checkpoint overhead. We use one training seed, so training-seed uncertainty is not estimated. The published baseline and S500 have different training histories; their comparison concerns practical deployment under limited adaptation, not the isolated causal effect of backbone size.

3 Progress and results

The frozen local dataset split contains 384 training and 48 held-out trajectories (47,043 and 5,927 frames). Hold out the last ten percent of trajectories per task, rounding up. For checkpoint selection, evaluate twenty evenly spaced held-out frames per task with fixed flow-noise seeds and sample-weighted mean loss. Preserve the published baseline normalization statistics across all arms.

Variant	Success [95% CI] (%)	Median / p95 (ms)	VRAM (GiB)	Weights (GiB)
B16	95.0 [92.7, 96.6]	79.1 / 84.3	5.22	5.15
Q8	94.8 [92.5, 96.4]	157.4 / 161.4	4.00	3.84
Q4	95.8 [93.7, 97.2]	93.6 / 97.9	3.29	3.20
S500	2.2 [1.2, 3.9]	1269.7 / 1277.8	3.07	2.14

Table 1: T1: final matched measurements. Success values use the reserved final initial states; trial counts are recorded in the accompanying results. Latency includes outer observation preparation and action postprocessing; VRAM is peak framework allocation. Weight size uses a consistent safetensors encoding with shared tensors deduplicated. Missing results remain pending. No adaptation is applied to B16/Q8/Q4 in this study. S500 measurements use the checkpoint at step 10,000.

The frozen adaptation budget is complete. Held-out loss selected step 10,000 (loss 0.254); simulator scores were not used for selection.

Measured results. Q8: 23.3% lower peak allocated memory, $1.99\times$ baseline latency; Q4: 36.9% lower peak allocated memory, $1.18\times$ baseline latency; S500: 41.1% lower peak allocated memory, $16.04\times$ baseline latency. These are deployment measurements on one GPU. Neural-only median/p95 latencies (ms) are B16 71.7/75.6, Q8 152.0/161.4, Q4 85.9/90.1, S500 234.5/235.4. Each neural path first reproduced native outputs on all 100 bank observations under identical flow-noise seeds. Final paired success differences from B16 (percentage points; 95% bootstrap intervals) are Q8 -0.2 [-1.2, +0.8]; Q4 +0.8 [-0.6, +2.2]; S500 -92.8 [-94.8, -90.6].

The companion repository provides checkpoint hashes, the delivery checklist, raw measurements, and reproduction commands. CUDA checks and finite training losses do not establish manipulation success; missing measurements are never imputed.

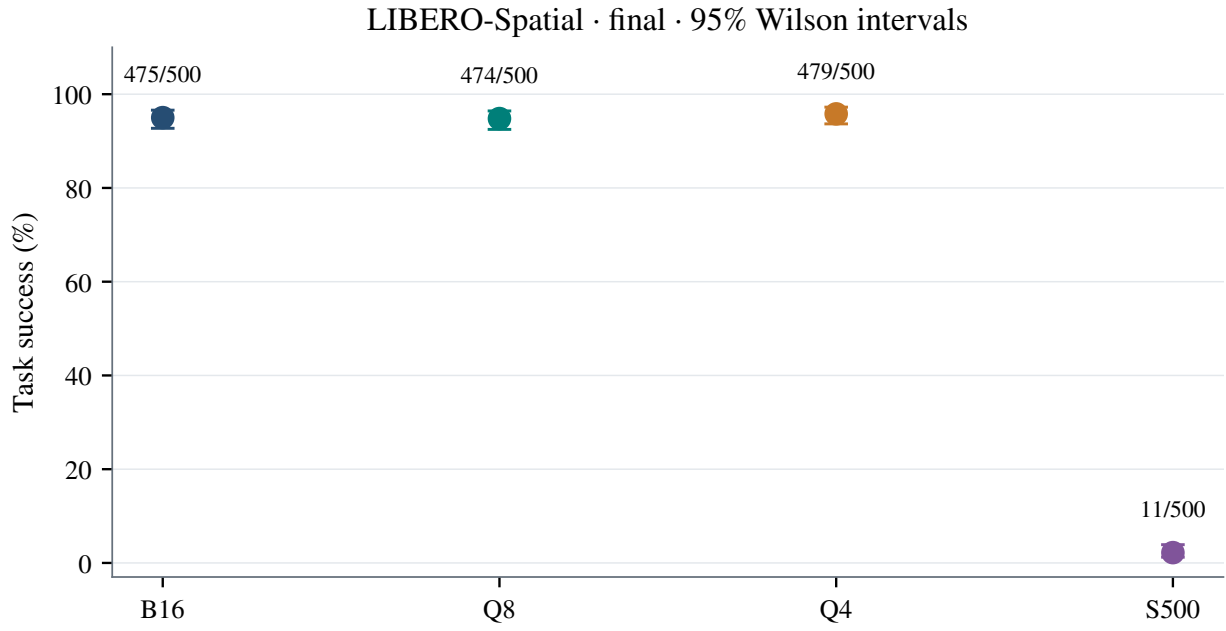


Figure 2: Manipulation success on the recorded rollout phase; intervals describe the fixed task suite.

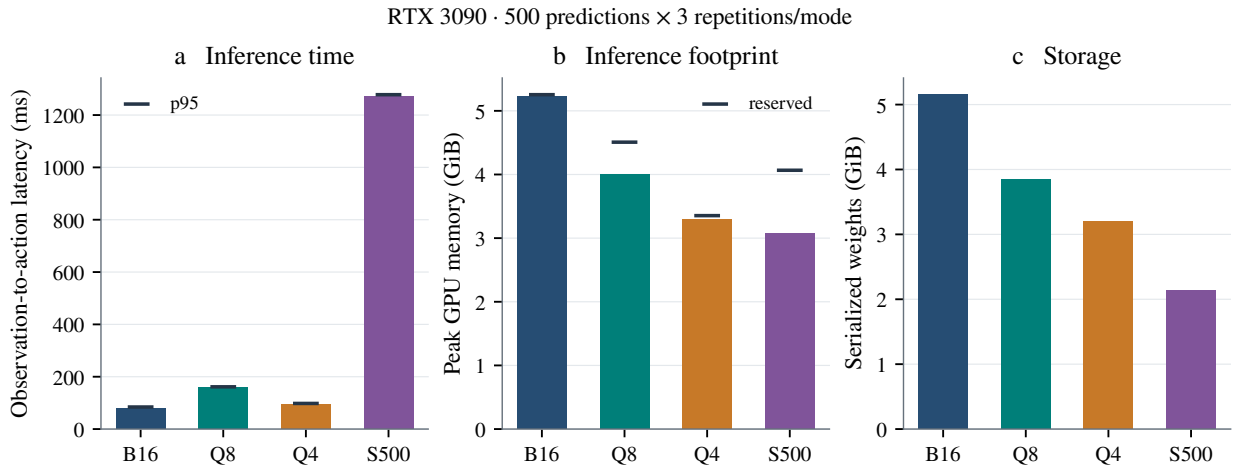


Figure 3: Measured deployment resources on the RTX 3090. Bars show median full observation-to-action latency, peak allocated memory, and safetensors weight size; ticks show p95 latency and peak reserved memory. Batch size one; simulator stepping is excluded.

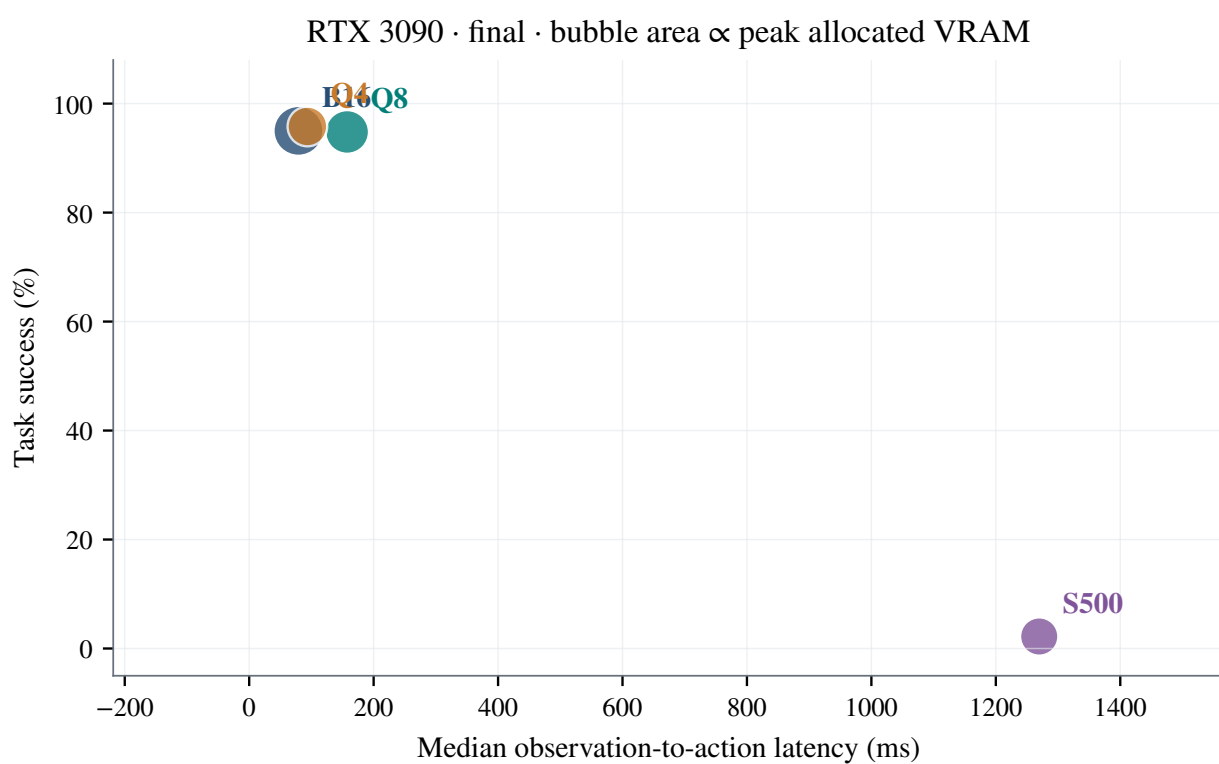


Figure 4: Measured success–latency trade-off. Marker area represents peak allocated GPU memory.

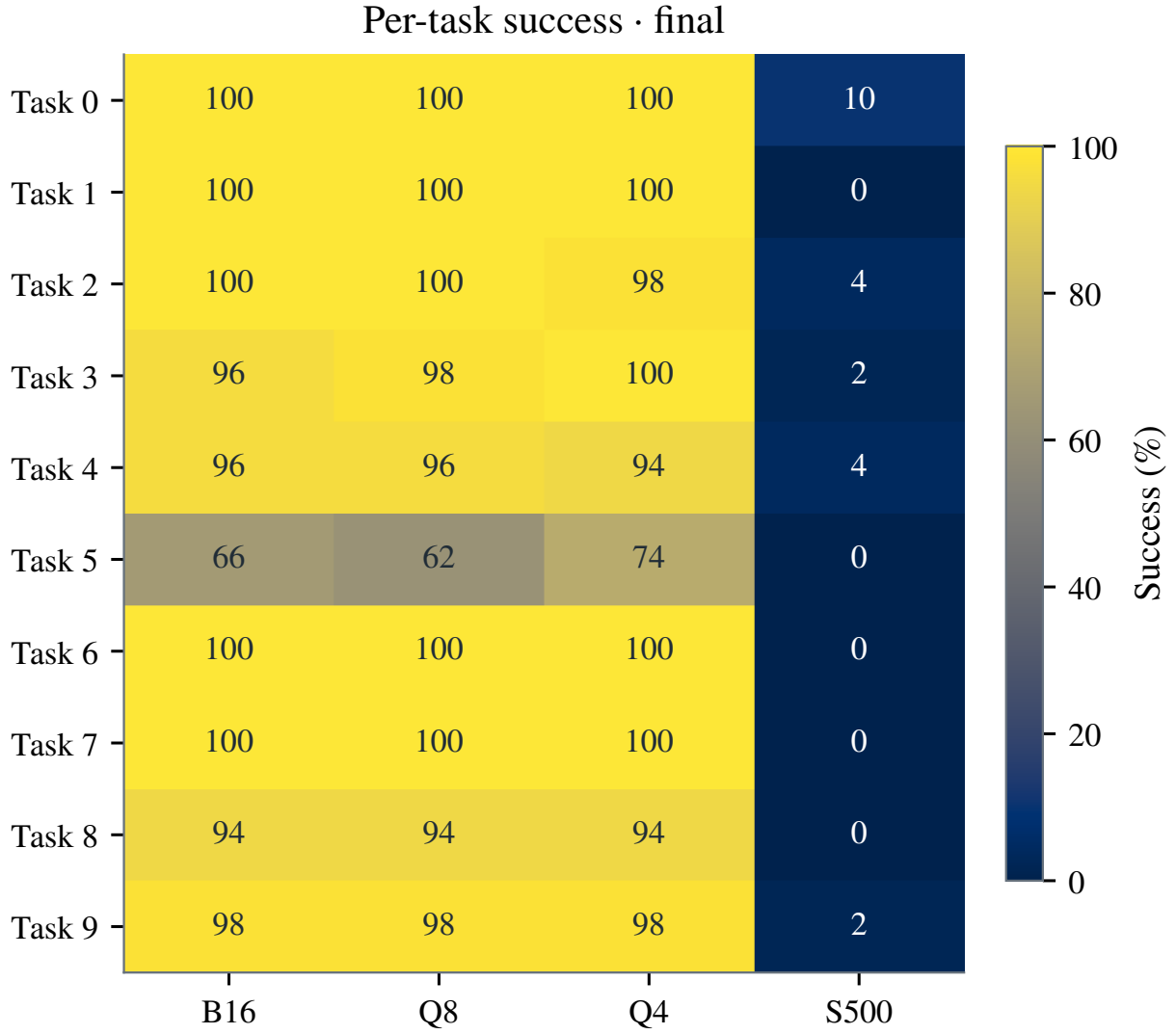


Figure 5: Task-level manipulation success; task numbers follow the committed initial-state manifest.

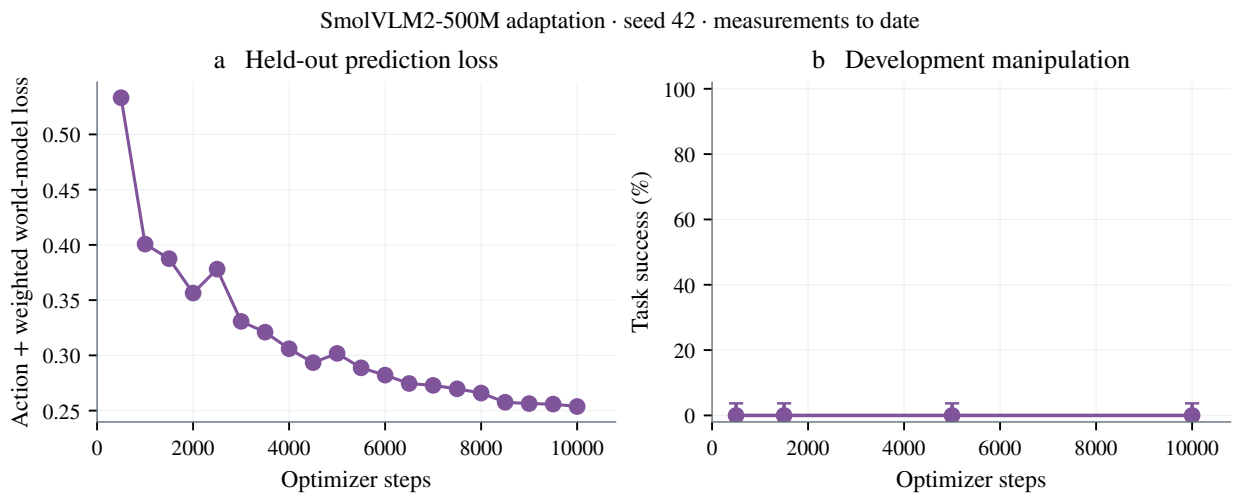


Figure 6: Adaptation measurements from the continuing training run. Missing development rollouts are marked explicitly; prediction loss is not robot success.

4 Decision rule and limitations

The prespecified engineering target is at least 25% lower peak inference memory with at most a five-percentage-point success drop from B16. Estimates are reported with uncertainty; an inconclusive interval does not establish performance preservation. Retaining the world model consistently prevents module removal from masquerading as a quantization benefit. A single GPU, limited adaptation budget, and differing pretraining histories limit broader conclusions.

The inherited world-model loss uses a bidirectional video encoder to produce both context and shifted targets (the checkpoint’s noncausal setting). Its loss is not an evaluation of causal future prediction. Video context enters only the training-time predictor; deployment receives current observations. Validation trajectories are held out from this adaptation run, not necessarily from published models’ earlier training. Checkpoint loading audits all mismatches: only verified tied-weight aliases and four unused checkpoint-only predictor state/extrinsics tensors are omitted.

Camera observations and outer policy preprocessing are matched, but each backbone retains its native image/token processor. A CPU probe confirms that SmolVLM expands a 224-pixel image into seventeen 512-pixel tiles. Thus the backbone comparison includes different visual token workloads; parameter count alone does not predict latency. Native pixel scaling was checked independently. Neural-only timing is separated from processor-inclusive timing.

Summary. This study measures decoder quantization within a fixed pretrained policy and backbone replacement under limited adaptation. Deployment savings and manipulation retention are separate outcomes; conclusions about successful robot deployment require both.

References

- [1] Sun et al. VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model. arXiv:2602.10098, 2026.
- [2] Liu et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. arXiv:2306.03310, 2023.
- [3] Marafioti et al. SmolVLM: Redefining Small and Efficient Multimodal Models. arXiv:2504.05299, 2025.
- [4] Assran et al. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985, 2025.
- [5] Dettmers et al. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339, 2022.
- [6] Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314, 2023.